

CAB BOOKING(Cab-U)

Full-Stack Cab Booking Management System (MERN)

Date: 6 July 2026

1. Introduction

Project Title: CAB BOOKING (FULL – STACK MERN)

Project Nature: A comprehensive full-stack cab booking platform built using the MERN stack. It connects passengers with drivers in real-time, providing a seamless interface for ride requests, fare estimation, and fleet management.

Team Members:

1. **HEMANTH SIVA SAI KAKI** – Full Stack Developer (Frontend, Backend, Database, Testing & Documentation).

2. Project Overview

Purpose: To provide a reliable and user-friendly digital cab booking platform that connects passengers with nearby drivers. The system manages the complete ride lifecycle, including user registration, ride booking, driver assignment, trip tracking, fare calculation, payment, and ride history.

Key Features:

- User Registration and Secure Login (Passenger, Driver, Admin).
- Real-time Ride Booking with Nearby Driver Assignment.
- Dynamic Fare Calculation based on distance and vehicle type.
- Live GPS Tracking and Route Navigation using Maps.
- Ride History, Booking Details, and Digital Trip Records.
- Multiple Payment Options (Cash, UPI, Cards - Future Enhancement).
- Admin Dashboard for managing users, drivers, and bookings.
- Driver Profile and Availability Management.
- Ride Status Updates (Requested, Accepted, Ongoing, Completed, Cancelled).
- Responsive Web Interface built using the MERN Stack.

3. Architecture

Frontend (React.js)

Built using React 18 for the frontend and Node.js with Express.js for the backend. MongoDB is used for data storage, and the application provides user authentication, cab booking, and ride management through a responsive web interface..

Components include:

- Home Page
- Login
- Signup
- Book Ride
- Ride History
- User Dashboard
- Admin Dashboard

Backend (Node.js & Express.js)

Backend is developed using

- Node.js
- Express.js

Backend provides REST APIs for

- Authentication
- Ride Booking
- User Management
- Driver Management
- Payments

Security includes

- JWT Authentication
- bcrypt Password Encryption

Database (MongoDB)

Backend is developed using

- Node.js
- Express.js

Backend provides REST APIs for

- Authentication
- Ride Booking
- User Management
- Driver Management
- Payments

Security includes

- JWT Authentication
- bcrypt Password Encryption

4. Setup Instructions

Prerequisites

- Node.js
- MongoDB Atlas
- VS Code
- Git
- npm

Installation

Clone Project

```
git clone https://github.com/username/ride-ready.git
```

Install Frontend

```
cd client  
npm install
```

Install Backend

```
cd server  
npm install
```

Environment Variables

Create .env

PORT=5000

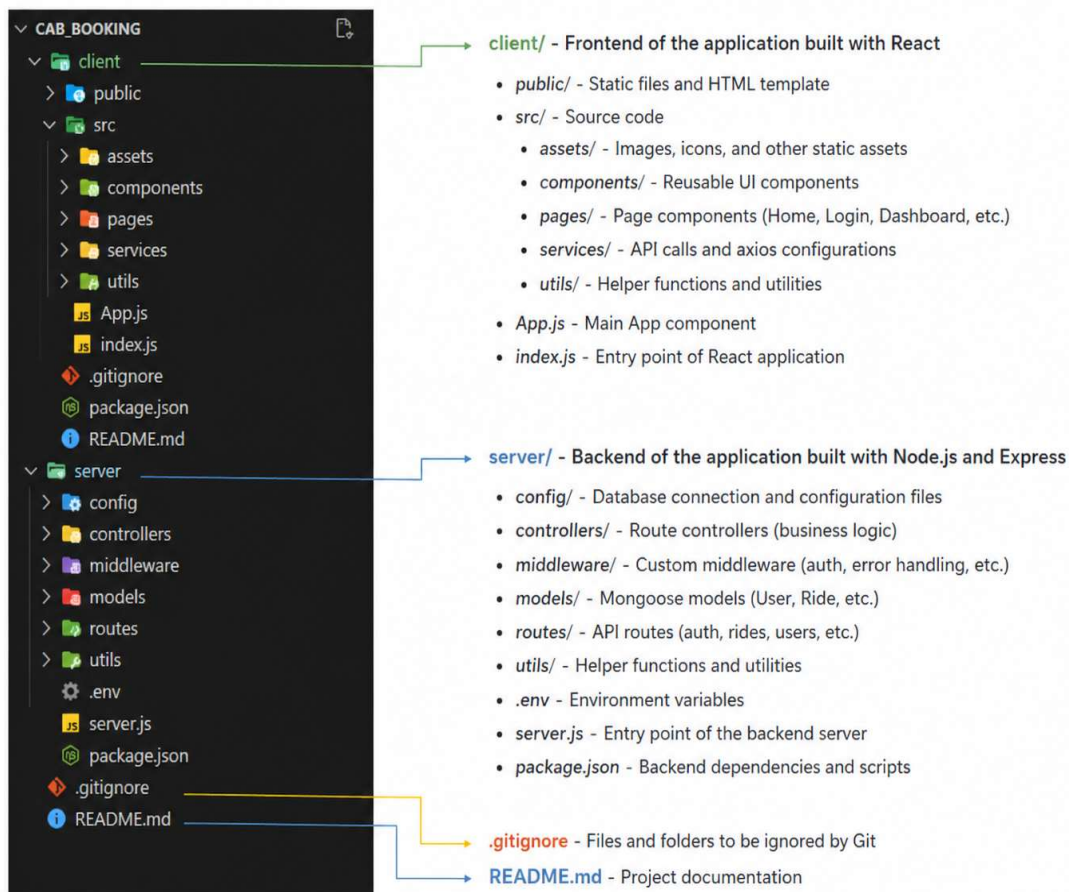
MONGO_URI=mongodb+srv://hemanthsivasai19_db_user:Hemanth19@cluster
0.2xwanud.mongodb.net/cab-u

JWT_SECRET=RideReadySecret

5. Folder Structure

The project follows a well-organized folder structure by separating the frontend and backend into independent modules. The frontend contains UI components and pages, while the backend manages APIs, routes, controllers, models, and database connectivity, making the application easy to maintain and scale.

Folder Structure



6. Running the Application

To run the Cab Booking (Cab-U) application locally, start both the backend and frontend servers in separate terminal windows.

Backend Server

1. Open a terminal.
2. Navigate to the server directory:
`cd server`
3. Install dependencies (first time only):
`npm install`
4. Start the backend server:
`npm start`
5. The backend server will run at:
`http://localhost:5000`

Frontend Server

1. Open a new terminal.
2. Navigate to the client directory:
`cd client`
3. Install dependencies (first time only):
`npm install`
4. Start the frontend application:
`npm start`
5. The frontend application will run at:
`http://localhost:3000`

7. API Documentation

The Cab Booking (CAB-U) system uses RESTful APIs to enable communication between the React frontend and the Node.js/Express backend. These APIs handle user authentication, ride booking, ride management, driver operations, and data retrieval securely and efficiently.

Method	Endpoint	Description
POST	/api/auth/register	Register a new user account.
POST	/api/auth/login	Authenticate user and generate login session.
POST	/api/rides/book	Book a new cab ride.
GET	/api/rides/history	Retrieve the user's ride history.
GET	/api/drivers	Fetch available drivers for booking.
PUT	/api/rides/:id/status	Update ride status (Accepted, Ongoing, Completed).
DELETE	/api/rides/:id/cancel	Cancel a booked ride.
GET	/api/users/profile	Retrieve logged-in user profile information.
PUT	/api/users/profile	Update user profile details.

8. Authentication & Role-Based Security

Protected by **JWT**. Differentiation between 'Passenger' and 'Driver' is handled at the middleware level. Payments are secured using **Stripe** integration or secure Wallet systems.

Authentication uses

- JWT Token
- bcrypt

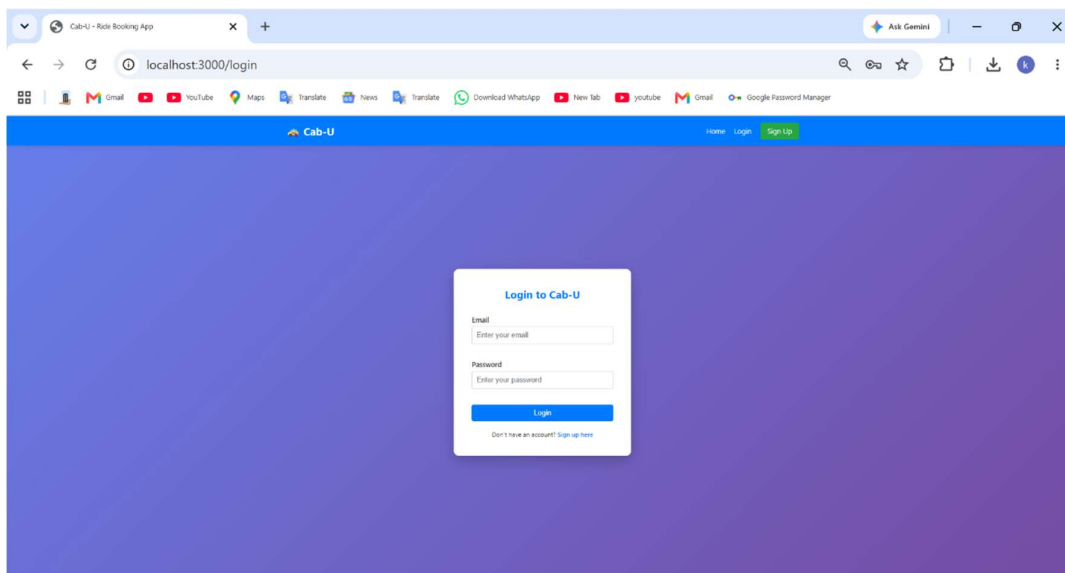
Workflow:-

```
graph TD
    A[1 User registers] --> B[Password encrypted]
    B --> C[Saved into MongoDB]
    C --> D[User Login]
    D --> E[JWT Token generated]
    E --> F[Token sent to frontend]
    F --> G[Protected APIs accessed using JWT]
```

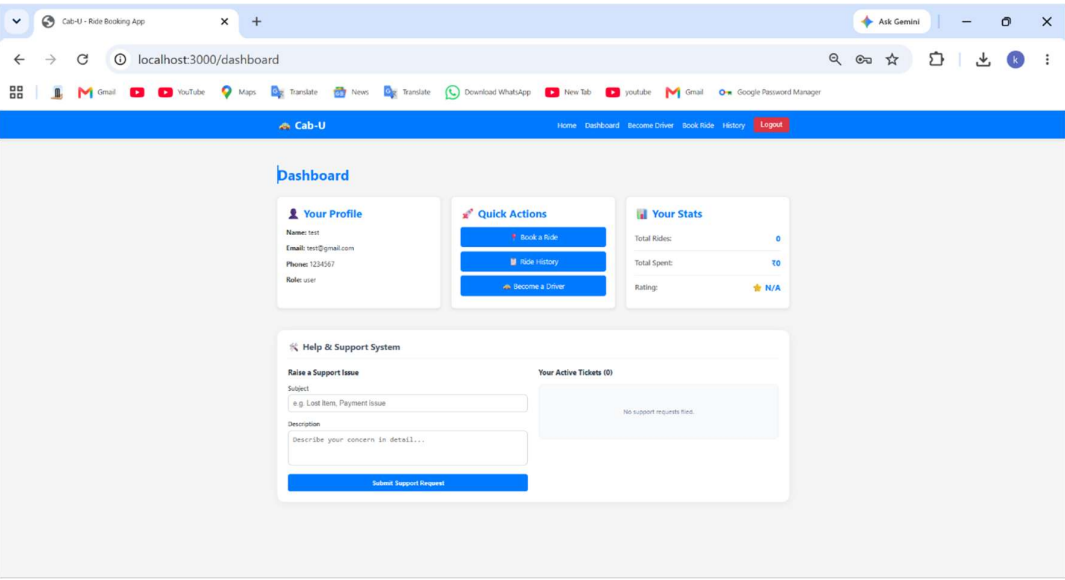
9. User Interface

The Ride Ready user interface is designed using **React.js** with a clean, responsive, and user-friendly layout. It provides separate screens for user registration, login, ride booking, ride tracking, payment, and admin management, ensuring a smooth device.

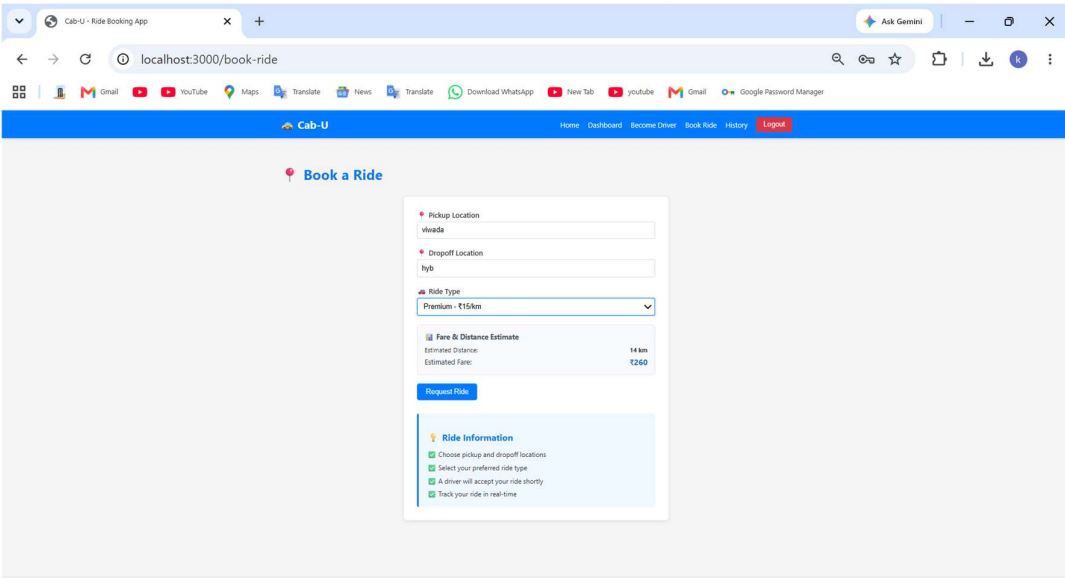
LOGIN:-



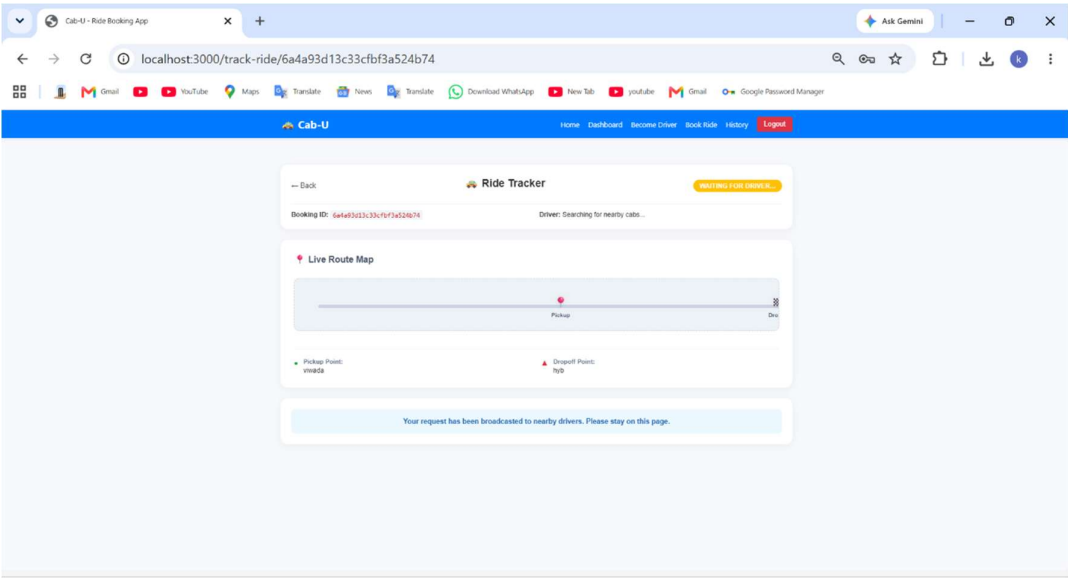
Dashboard :-



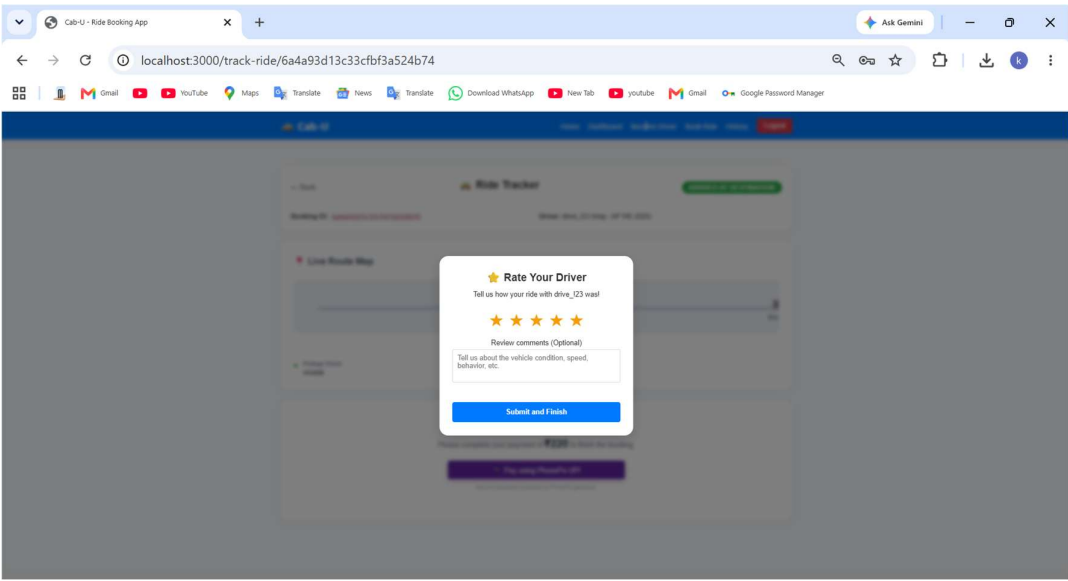
RIDE BOOKING:-



RIDE TRACKING :-



REVIEW & PAYMENT OF THE DRIVE:



10. Testing & Validation

Testing includes

- User Registration Testing
- Login Testing
- Ride Booking Testing
- Payment Testing
- Driver Assignment Testing
- Authentication Testing

Testing Type

- Functional Testing
- Manual Testing
- API Testing

Expected Result

All modules work successfully.

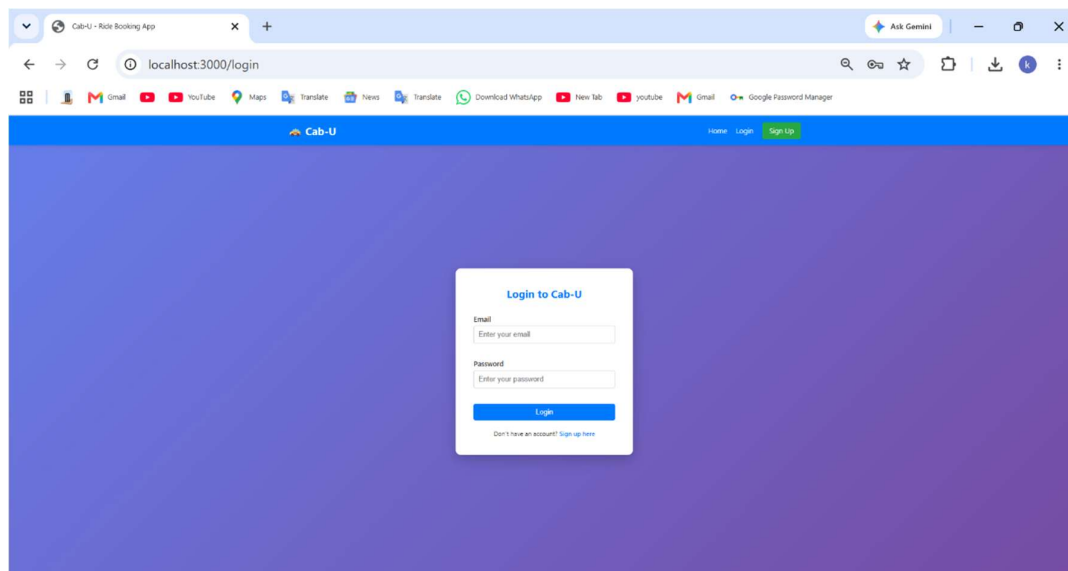
11. Screenshots or Demo

Demo Link:-

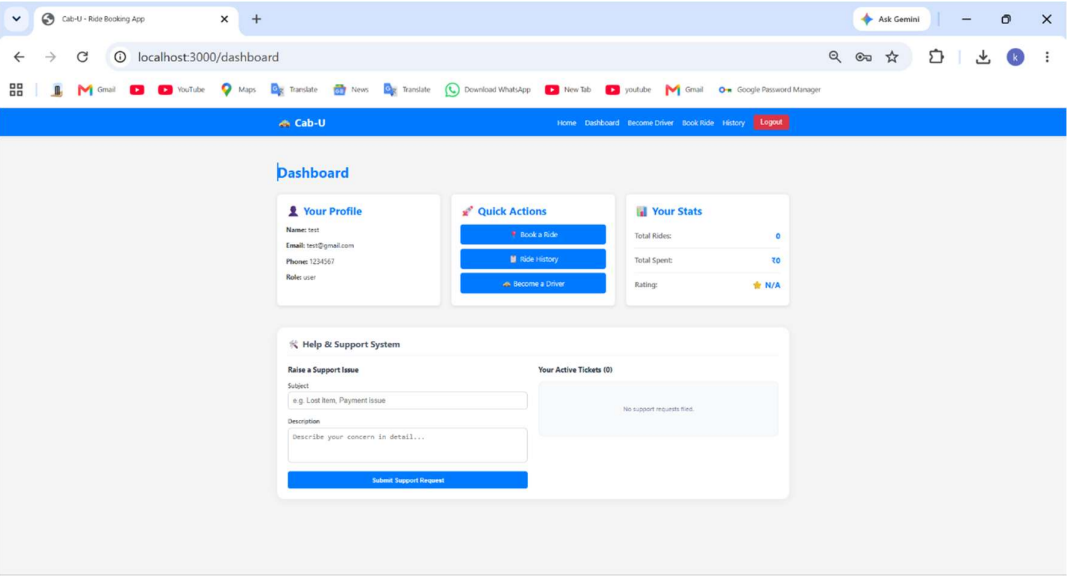
<https://github.com/Hemanth-1912/CAB-BOOKING>

Screenshots:-

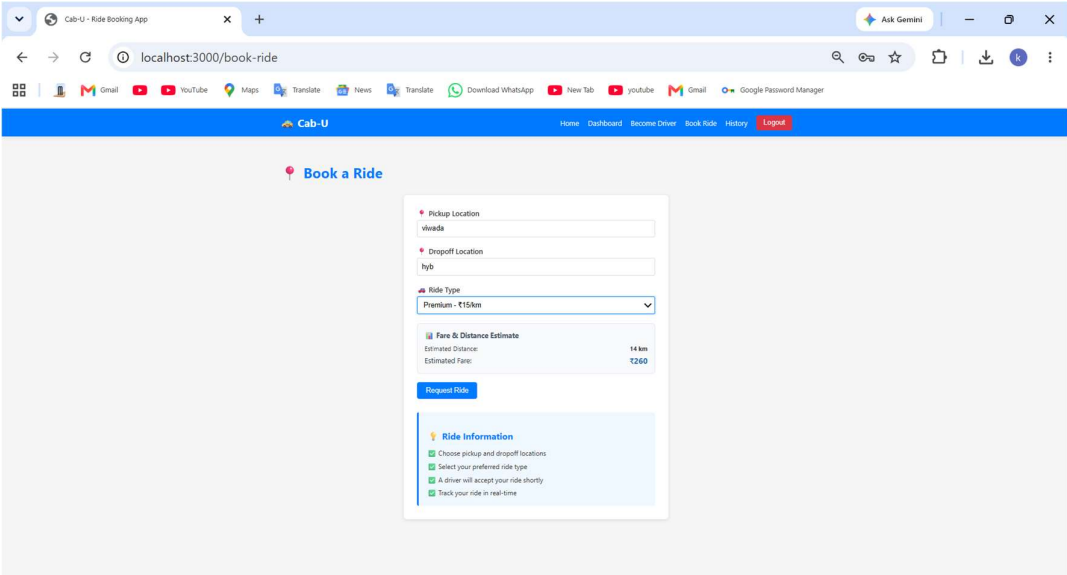
LOGIN:-



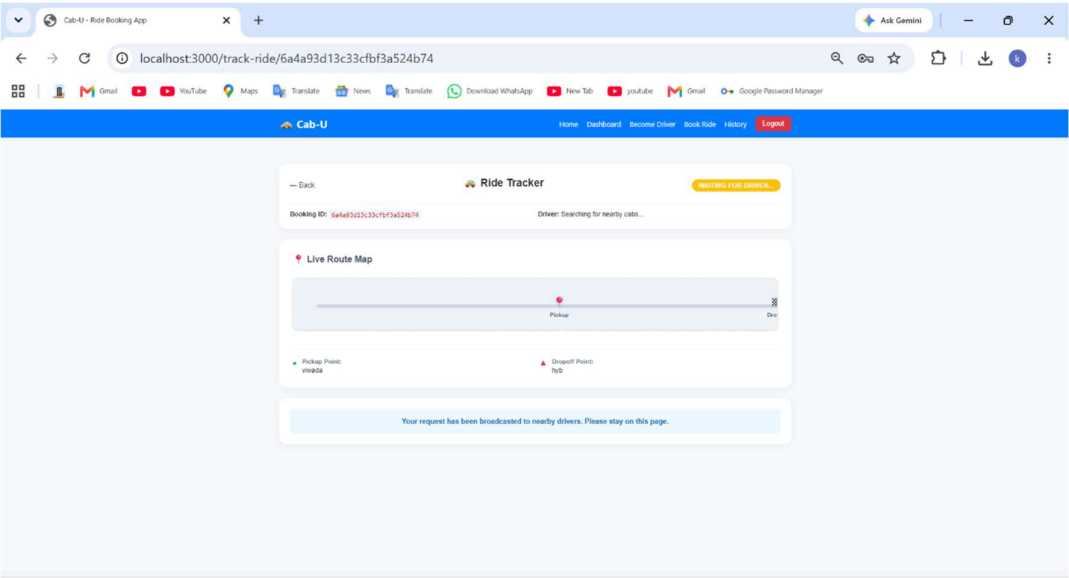
CAB BOOKING Dashboard :-



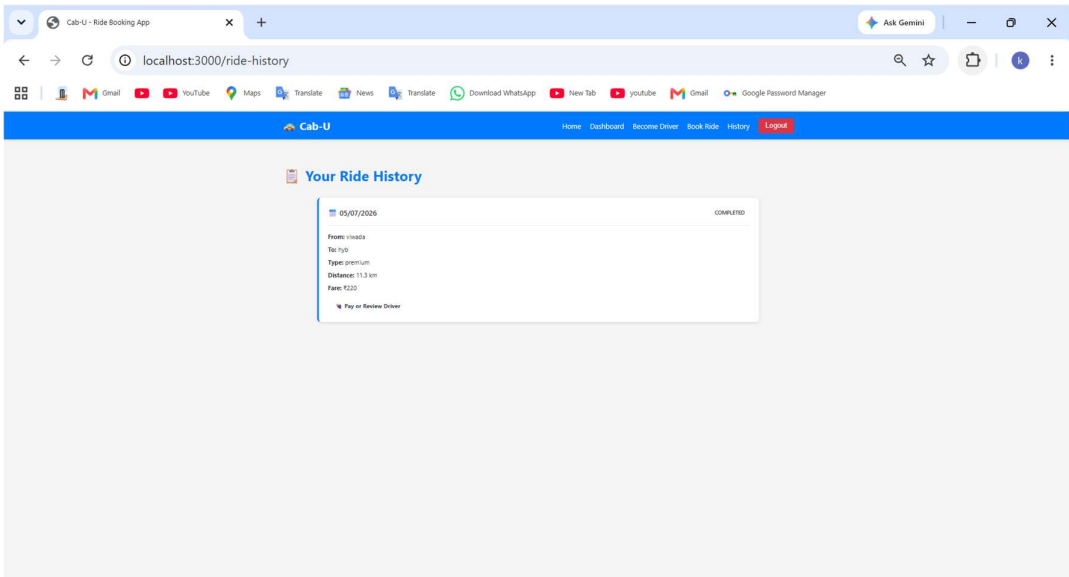
RIDE BOOKING:-



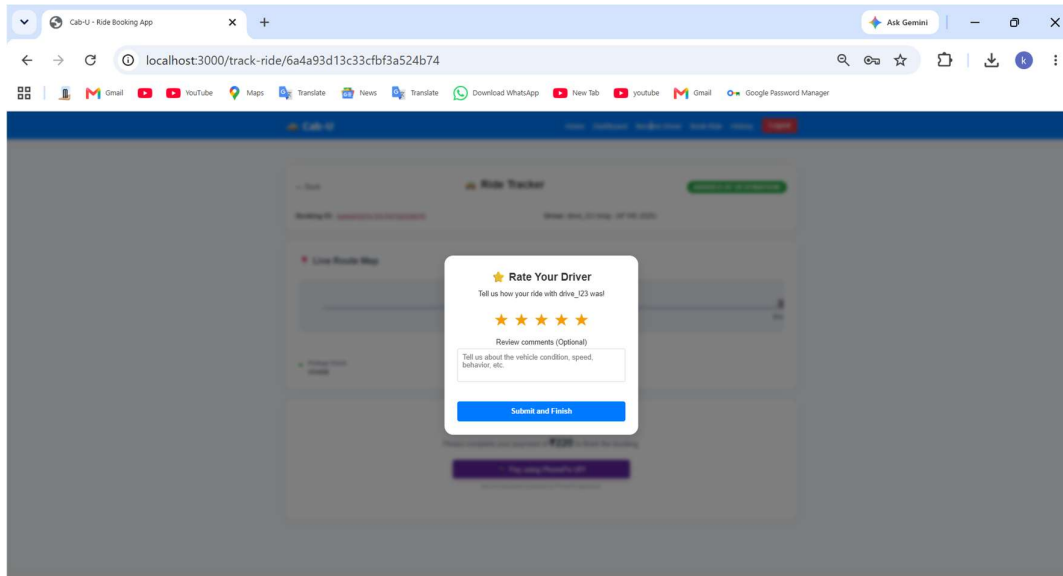
RIDE TRACKING :-



RIDE HISTORY:-



REVIEW & PAYMENT OF THE DRIVE:-



12. Known Issues

- GPS accuracy may reduce in areas with poor internet or weak network coverage.
- Live GPS tracking is currently limited and does not provide continuous real-time updates.
- Online payment gateway integration is under development and not fully functional.
- Driver availability status may experience slight delays due to network latency.
- Push notifications for ride booking and ride status updates are not yet implemented.
- The application currently supports only web browsers and does not include a dedicated mobile app.
- Ride scheduling for future bookings and advanced fare calculation features are planned for future releases.

13. Future Enhancements

Future Enhancements

- Implement **real-time GPS tracking** to provide accurate live driver and ride location updates.
- Integrate **secure online payment gateways** such as Razorpay, Stripe, and UPI for seamless digital transactions.
- Add **push notifications** to keep users informed about ride confirmations, driver arrivals, and payment status.
- Introduce **ride scheduling** to allow users to book cabs in advance for future travel.
- Develop a **driver rating and customer feedback system** to improve service quality and user satisfaction.
- Integrate **AI-based fare estimation and smart driver allocation** for faster ride matching and optimized pricing.
- Launch **Android and iOS mobile applications** with multilingual support and SOS emergency features for enhanced accessibility and safe.